279 自然言語処理支援エディタの改良－単語辞書の階層化－

堤 正勝† 荒木 健治†† 宮永 喜一† 栃内 香次†
†北海道大学工学部 ††北海学園大学工学部

１．はじめに

自然言語処理研究における実験システムの作成において、幾つかのツールが有効である。しかしながら、統合環境として実験システムを作成する支援が行なえるものとして充分な機能を持ったシステムはまだないといえる。そこで、我々はこの様な機能を備えた環境をエディタとして作成し、報告してきた[1][2]。本稿ではそのエディタの機能の充実を図るひとつの試みについて報告する。単語辞書を階層化することにより、かな漢字変換などの当てはめコストを減少できることは既に報告されている[3]。本稿では、対象とする分野ごとに、辞書を細分化し、優先順序を入れ換えることで、さらに語の選択精度を向上させること、また、将来的には意味などによる単語限定が可能な構造を本エディタに追加したのでこれを報告する。

２．自然言語処理向きエディタとは

本エディタは自然言語処理向きエディタとして、Unix 上でＣ言語を用いて作成したエディタである。テキストのロード時に形態素解析を行ない、その後の自然言語処理に適した単語単位に分割した構造に内部展開する。そのために、本エディタでは辞書を起動時に主記憶にロードし、既存テキストならば読み込み時に形態素解析を行なって、単語と認定された文字列は全て辞書の単語エントリへのポインタで表現する。ユーザインターフェースに関しては、emacs 指向で、ファイル管理機能（ディレクトリエントリの一覧を表示して、編集、削除、ディレクトリ変更、名前変更などを行なわせる）、圧縮ファイル管理機能などをもつ。さらに、実験システム開発支援として、自然言語処理に関するサービスルーチン、表示管理機能（カラー対応）、デバッグ用バッファなどをもつ。自然言語処理の実験システムを作成するにあたり、これらのエディタ内の機能を利用することにより、かなり作業が楽になると考えられる。なお、実験システムはＣ言語で記述し、エディタにリンクするという形をとる。なお、説明の都合上、本エディタはｚｅｅ（仮称）とよぶ。

３．辞書の階層化

多段階分割法によれば、少数の確実性の高い語から順に島駆動的に確定することで、より少ないコストで、高い変換率が得られることが報告されている。そこで、使用者や、分野により、単語の確実性が異なると考えられるので、必要に応じてこれらの順序を入れ換えることが考えられる。その際使用者毎に、さらに分野毎に辞書を作成していたのでは、使用者数×分野数の辞書が必要となり、スペース上あまり好ましくない。そこで、個々の単語の確実性を入れ換えるのではなく、ある程度の語群（サブ辞書）を単位に順序の入れ換えを行なうことで対処する。このような複数辞書にまたがる、検索、登録、削除、移動、複写などの機能を追加している。

３．１ 構造化辞書エディタ／マネージャ（ｄｍ）

エディタｄｍは例えば、ファイルシステムにおけるサブディレクトリを作成し、そこにファイルを移動する様なイメージで、単語を分類するエディタである。現在は、プロトタイプとして、上記エディタｚｅｅとは別に作成しているが、将来的には、エディタｚｅｅの方に移植する予定である。本エディタｄｍは、既存の辞書（パソコン、ワープロのフロントエンドや、Unix のかな漢字変換サーバで使用されている辞書など）を外部のプロセス（フィルタ）で括弧形式に変換したものをロードする。どの辞書が、どのフィルタを使用するかは、リソースファイルを用いて記述する。フィルタは、著作権などの問題があるので、基本的には各自作成してもらうものとする。本エディタでは、どのサブ辞書をどういう順序で使用するかという情報を作成し、文脈名をつけることができるようになっている。

３．２ 辞書の使用

エディタｚｅｅでは構造化辞書を読み込み、単語は単語辞書へ、構造は構造リストへ分離し、構造リストからポインタで辞書内を参照する形でて管理する。新規編集ファイルでは、文書入力モードを指定したあと、ｄｍで作成した文脈名を選択するようにして、使用する辞書の順序を決める。かな漢字変換や、誤り訂正などの単語検索は、この順序で検索が行なわれる。

なお、現在、エディタｄｍの機能拡充、フィルタ整備、構造化辞書編集を行なっている最中である。

４．エディタに関する質問

エディタｚｅｅに関してこれまでに問題になった点とその検討結果をここにまとめておく[1][2]。

４－１．なぜ emacs 上に作成しないのか

メモリ制限枠がある。また、Lisp であるためにメモリ効率が悪い。データ構造の制約がある。見掛け上構造体の様なものを扱えても実際にはλバインドをたぐる必要があるので、ＣＰＵ効率が悪い。Unix ではＣ言語がシステム言語であり、Ｃはわかるが、Lisp は知らないというユーザも少なくない。Lisp であるので、GC が勝手に発生する。

４－２．サーバ・クライアント・モデル

辞書をサーバ・クライアント・モデルにすべきではないかという質問がある。 形態素解析の都合により全ての単語を検索対象とするため、結局全ての単語をサーバから取り出すことになり、あまり効率的にはならない。ＮＦＳがあれば充分である。辞書アクセスのコストが上がると、通信コストが上がるので不利である。また、既存サーバとは異なる文法体系をインプリメントしたい場合に不都合がある。

４－３．ユーザインターフェース

過去の報告で、エディタの機能を紹介すると必ず問われる問題である。他のエディタにない機能をいろいろ付加することにより入力の効率化を図ってはいるが、これは主たる目的ではなく、このエディタの構造ならでは、付加できる機能であり、利便機能に過ぎない。本質は本エディタを実験システムの開発に用いることにより、繁雑な入力、表示インターフェースの記述の簡略化、が行なえるというところに重点がある。よって、他のエディタと比べて、これこれの機能のために、どのくらい時間の節約になった、等の評価はしない。

５．おわりに

本稿では、自然言語処理向きエディタに階層構造を持つ辞書を追加しているのでこれについて報告した。現在、構造化辞書エディタｄｍを作成し、分野別辞書を編集中である。発表時には、操作例などをつけて報告したい。今後の予定としては、本エディタの有効性を何等かの形で示さなければならないので、情報処理学会に報告されている自然言語処理の事例をいくつかインプリメントしてみようと考えている。

参考文献

[1] 堤 正勝、荒木健治、宮永喜一、栃内香次：
自然言語処理支援エディタの改良，
平成６年度電気関係学会北海道支部連合大会，室蘭工大,1994

[2] 堤 正勝、荒木健治、宮永喜一、栃内香次：
自然言語処理研究向き文書エディタの試作について，
信学技報,SS-94-37,pp17-24,1994-11.

[3] 堤 正勝、荒木健治、宮永喜一、栃内香次：
多段階分割復元法を用いた連続音声認識の誤りの復元，
電子情報通信学会，音声研究会資料，SP89-92,1989.